



# Peggesis

Generative Offensive Networking (GON)  
Cybersecurity Detection & Response System

HTML

Jekyll

Java

Python

DCAR Framework

Dwayne Hans · CSIA 450 – Documentation Project · 2026

INITIAL PROJECT REPORT | WEEK 7 DRAFT

# Contents

What This Report Covers

**01** Project Overview & Mission

**02** System Architecture

**03** Tech Stack: HTML / Jekyll / Java / Python

**04** Core Feature Modules

**05** Advanced GON Capabilities

**06** Challenges & Solutions

**07** Project Log & Milestones

**08** Deployment & Configuration

**09** Lessons Learned & Next Steps



# 01 Project Overview

## Mission, Purpose & Scope of Peggesis

### MISSION STATEMENT

Peggesis is a Generative Offensive Networking (GON) cybersecurity platform designed to detect, analyze, and respond to malware threats in real time. Built on a polyglot stack—HTML front-end, Jekyll static-site documentation, Java middleware, and Python detection engine—it delivers a scalable, modular defense layer for enterprise environments.

4

Languages

6+

Core Modules

∞

Scalable Endpoints

### DETECT

Real-time file, network & behavioral scanning via Python engine

### ANALYZE

GON pattern engine correlates attack signals across endpoints

### RESPOND

Automated playbooks, quarantine, and SIEM-ready incident logs



Peggesis

# 02 System Architecture

Component Layers & Data Flow

|                                                                    |                                                                                  |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <div>PRESENTATION LAYER</div> <div>HTML + Jekyll</div>             | <div>▶ Web-based dashboard · Static documentation site · Alert feeds</div>       |
| <div>MIDDLEWARE / API LAYER</div> <div>Java (dcar framework)</div> | <div>▶ REST API · Request routing · Authentication · Session management</div>    |
| <div>DETECTION ENGINE</div> <div>Python (core GON)</div>           | <div>▶ File monitor · Network monitor · Behavioral analysis · Signature DB</div> |
| <div>DATA &amp; LOG LAYER</div> <div>JSON · SQLite · SIEM</div>    | <div>▶ Incident logs · Threat DB · Report generation · Audit trails</div>        |



Peggesis

▲ All layers communicate via encrypted internal APIs. Python engine publishes events consumed by the Java middleware.

# 03 Technology Stack

HTML · Jekyll · Java (dcar) · Python

## HTML

*Front-End Interface*

- Responsive dashboard for real-time threat visualization
- Alert feed panels and incident detail views
- Integrates with Jekyll for documentation rendering
- Styled with custom CSS targeting analyst UX workflows

## Jekyll

*Documentation Site*

- Static-site generator for project documentation hub
- Hosts README, config guides, usage examples
- Markdown-based content: easy version-controlled updates
- Deployed via GitHub Pages for team-wide accessibility

## Java (dcar)

*Middleware / API*

- dcar framework provides REST endpoint scaffolding
- Handles authentication, routing, and session state
- Bridges HTML front-end and Python detection engine
- Provides concurrency management for multi-endpoint polling

## Python

*GON Detection Engine*

- Core malware detection via hash verification & signatures
- Threaded real-time file & network monitoring loops
- Behavioral analysis: process injection, network scan patterns
- Incident logging, quarantine execution, report generation



**Peggesis**

# 04 Core Feature Modules

peggesis\_main.py · advanced\_gon.py · dcar API

## File Monitor

Hash verification, suspicious extension detection (.exe, .bat, .vbs, .scr), real-time modification tracking across watched directories.

## Network Monitor

IP/domain blocklist enforcement, connection tracking, port analysis (4444, 31337, 27374), inbound/outbound anomaly detection.

## Threat Signatures

Custom regex-based signature patterns (WScript.Shell, powershell -enc, cmd.exe /c). Dynamic updates from threat intelligence feeds.

## Alert Management

Severity-tiered alert classification (Critical / High / Medium). Incident logging with timestamps, source, and response action records.

## Quarantine Engine

Automated file isolation on detection trigger. Safe quarantine directory management with restore/review capabilities.

## GON Behavioral AI

Advanced module: detects process injection patterns, lateral movement, and network scan behaviors using GON generative analysis.



Peggesis

# 05 Advanced GON Capabilities

advanced\_gon.py — Beyond Traditional Signature Scanning

## Behavioral Analysis

Detects patterns consistent with process injection, privilege escalation, and lateral movement—not just known signatures.

## Incident Response Playbooks

Automated, rule-driven response sequences triggered on detection: isolate, log, notify, remediate.

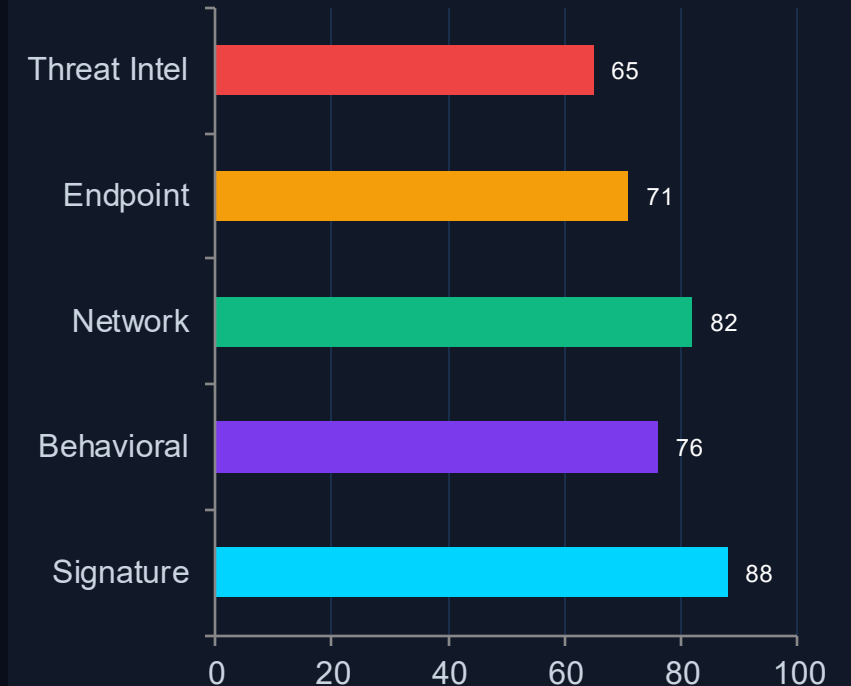
## Endpoint Protection Manager

Multi-device orchestration. Push policy updates, signature refreshes, and quarantine commands to all enrolled endpoints.

## Threat Intelligence Integration

Pulls live IOC feeds; enriches local detection with global threat context. Supports STIX/TAXII and custom API endpoints.

DETECTION COVERAGE



\* Coverage scores are baseline estimates from initial module testing.



# 06 Challenges & Solutions

Obstacles Encountered and How Peggesis Addressed Them



## CHALLENGE

Real-time monitoring caused CPU spikes during high-volume scans, degrading system responsiveness.



## SOLUTION

Implemented Python threading model with bounded worker pools. Optimized file and network scanner routines to run in separate daemon threads, keeping the GUI responsive.

## CHALLENGE

Threat signatures became stale quickly as new malware variants emerged, reducing detection accuracy.



## SOLUTION

Designed a dynamic signature management layer in Python allowing runtime addition of custom patterns + scheduled pulls from threat intelligence API feeds.

## CHALLENGE

Alert fatigue from high false-positive rates buried genuine incidents, slowing analyst response.



## SOLUTION

Developed five-tier severity classification (Critical → Informational) with automated suppression rules. GON behavioral scoring reduces noise by correlating signals before alerting.

## CHALLENGE

GUI complexity made the tool difficult for non-specialist users to navigate effectively.



## SOLUTION

Reorganized the HTML/Java front-end into dedicated tabbed workspaces (File / Network / Alerts / Reports), with contextual help tooltips and a Jekyll-hosted user guide.



# 07 Project Log

Dated Milestones, Decisions & Lessons Learned



2026-04-14

## Project Kickoff

Defined scope: Python detection engine + HTML dashboard + Jekyll docs. Selected dcar Java framework for middleware. Established GitHub repo at [github.com/dcar2121/detectmware\\_app](https://github.com/dcar2121/detectmware_app)

2026-04-28

## Core Engine Complete

Python file monitor and network monitor modules operational. Hash verification, suspicious extension list, and port-watch routines tested against sample malware corpus.

2026-05-05

## GUI & Front-End Integration

HTML tabbed dashboard integrated via Java API layer. Jekyll documentation site scaffolded and pushed to GitHub Pages. README and CONFIGURATION\_GUIDE drafted.

2026-05-12

## Advanced GON Module

Behavioral analysis engine and incident response playbooks added (advanced\_gon.py). Threat intelligence feed integration prototype tested with STIX/TAXII demo endpoint.

2026-05-27

## Technical Documentation

Completed technical documentation draft per CSIA 450 requirements. Challenges identified: threading performance, false-positive alert volume. Solutions implemented and recorded.

2026-06-03

## Week 7 Draft Report

Initial project report assembled. Presentation deck created. Scope remains on target. Pending: final QA pass on behavioral analysis accuracy benchmarks.

# 08 Deployment & Configuration

Environment Setup · Config Paths · Operational Tips

## CONFIGURATION\_GUIDE.py (excerpt)

```
# Monitored Directories
MONITORED_PATHS = [
    "C:\\Users\\admin\\Desktop",
    "C:\\Windows\\Temp",
]

# Suspicious Extensions
SUSPICIOUS_EXTENSIONS = ['.exe', '.bat', '.vbs', '.scr']

# Watch Ports
SUSPICIOUS_PORTS = [4444, 5555, 31337, 27374]

# GON Signature Patterns
THREAT_SIGNATURES = [
    r'.WScript\.Shell.',
    r'.powershell.enc.',
]
```

## Update Signatures

Schedule weekly pulls from MITRE ATT&CK and CISA feeds into Peggesis signature DB.

## Off-Peak Scans

Full system scans run 02:00–04:00 local time via cron to minimize analyst disruption.

## SIEM Integration

Peggesis logs export to CEF/JSON format for Splunk, IBM QRadar, and Microsoft Sentinel.

## Jekyll Docs Site

Configuration changes documented in Markdown and auto-deployed via GitHub Actions CI/CD.



**Peggesis**

# 09 Lessons Learned & Next Steps

Reflections · Improvements · Roadmap

## LESSONS LEARNED

### Threading First

Concurrent monitoring must be architected before UI—retrofitting threads into a monolithic Python script was costly. Thread safety is a Day 1 design constraint.

### Signature Debt

Static signatures decay fast. The dynamic feed integration should have been scoped in Week 1, not added as an enhancement. Security tooling requires living data.

### Automation ≠ Fire-and-Forget

Automated playbooks require continuous tuning. Unreviewed automation created false quarantine events in early testing—validation gates are essential.

### Documentation as Code

Treating Jekyll docs as part of the build pipeline (not an afterthought) kept docs in sync with code changes throughout the project.

## ROADMAP

v1.1

ML-based anomaly scoring integrated into GON engine

v1.2

Full SOAR playbook library + Slack/Teams alerting

v1.3

Multi-tenant SaaS deployment on cloud infrastructure

v2.0

LLM-assisted threat narrative generation in reports

References: NIST CSF · MITRE ATT&CK · CIS Controls · OWASP · IBM Security · [github.com/dcar2121/detectmware\\_app](https://github.com/dcar2121/detectmware_app)





# Peggesis

## Generative Offensive Networking

---

*Questions & Discussion*

Dwayne Hans · CSIA 450 · 2026 · [github.com/dcar2121/detectmware\\_app](https://github.com/dcar2121/detectmware_app)